



Alergia Algorithm: Simulation & Implementation

Automata Modeling for DNA Sequence Representation

Your Name

BUET

September 11, 2026

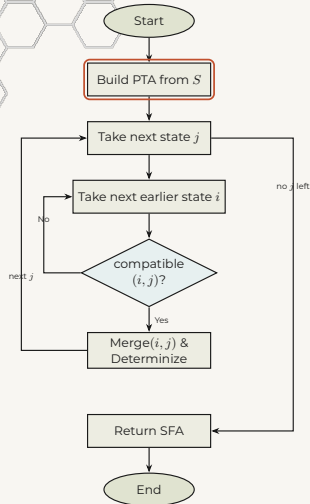
- Previous part: built a **Prefix Tree Acceptor (PTA)** directly from training strings
- The PTA is **rigid** — it accepts only the exact strings it was built from
- DNA sequences carry point mutations, so we need something that **generalizes**
- This part: the **Alergia algorithm** that turns the rigid PTA into a compact, fault-tolerant automaton — and a working simulation of it

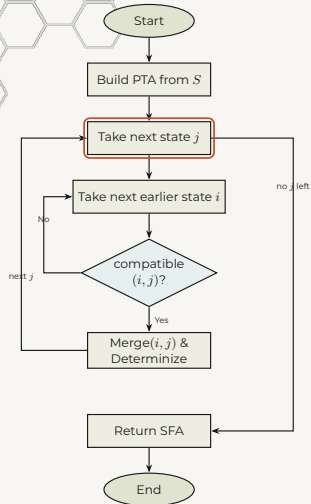
One-line summary

Compare pairs of states statistically; if their behavior is indistinguishable within a confidence bound, **merge** them. Repeat until no more merges are possible.

- Each state tracks: how many strings pass through it, how many take each next symbol, how many end there
- Two states are **compatible** if these statistics agree within a confidence parameter α
- Compatible states get merged, and their subtrees are folded together

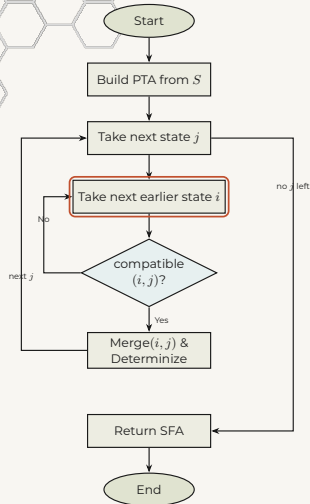
Input: S, α **Output:** SFA
 $A \leftarrow$ stochastic PTA from S





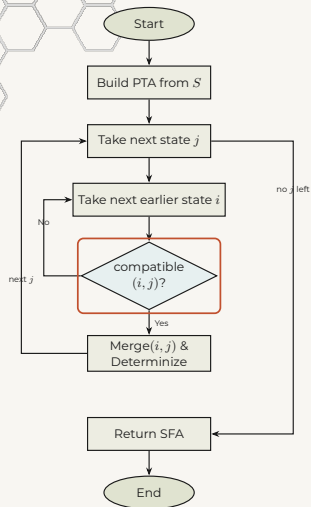
Input: S, α **Output:** SFA
 $A \leftarrow$ stochastic PTA from S

for $j = \text{successor}(\text{firstnode}(A))$ **to** $\text{lastnode}(A)$:



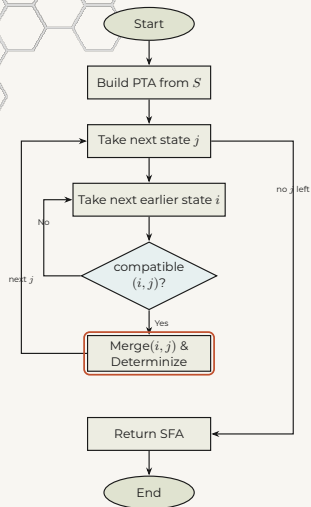
Input: S, α **Output:** SFA
 $A \leftarrow$ stochastic PTA from S

for $j = \text{successor}(\text{firstnode}(A))$ **to** $\text{lastnode}(A)$:
 for $i = \text{firstnode}(A)$ **to** j :



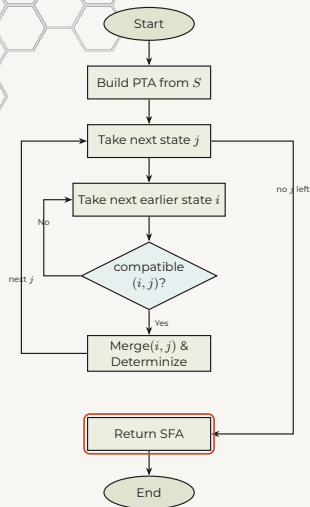
Input: S, α **Output:** SFA
 $A \leftarrow$ stochastic PTA from S

for $j = \text{successor}(\text{firstnode}(A))$ **to** $\text{lastnode}(A)$:
 for $i = \text{firstnode}(A)$ **to** j :
 if $\text{compatible}(i, j)$:



Input: S, α **Output:** SFA
 $A \leftarrow$ stochastic PTA from S

for $j = \text{successor}(\text{firstnode}(A))$ **to** $\text{lastnode}(A)$:
 for $i = \text{firstnode}(A)$ **to** j :
 if $\text{compatible}(i, j)$:
 $\text{merge}(A, i, j)$
 $\text{determinize}(A)$
 exit i loop



Input: S, α **Output:** SFA
 $A \leftarrow$ stochastic PTA from S

for $j = \text{successor}(\text{firstnode}(A))$ **to** $\text{lastnode}(A)$:

for $i = \text{firstnode}(A)$ **to** j :

if $\text{compatible}(i, j)$:

$\text{merge}(A, i, j)$

$\text{determinize}(A)$

 exit i loop

return A

Two states are compared using a Hoeffding-bound based confidence test:

$$\left| \frac{f}{n} - \frac{f'}{n'} \right| > \sqrt{\frac{1}{2} \log \frac{2}{\alpha} \left(\frac{1}{\sqrt{n}} + \frac{1}{\sqrt{n'}} \right)}$$

- n, n' = number of strings arriving at each state
- f, f' = number of strings ending / following a given transition
- If the left side exceeds the right side, the states are **different** — otherwise they are merge candidates

Key takeaway

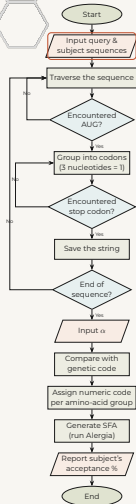
Lower $\alpha \rightarrow$ looser test $\rightarrow$ more merging $\rightarrow$ more generalization (more mutation-tolerant).

Compatibility is not just local — it is checked **recursively** down the subtree:

- States i and j must agree on ending-frequency and every symbol's transition frequency
- Their destination states (children) must *also* be compatible, all the way down
- Only if the whole subtree agrees are i and j merged

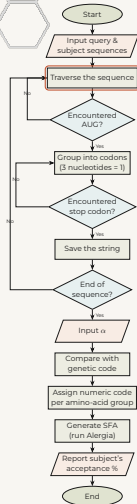
This is why merging two states can cascade into merging several more states at once.

THE FULL PIPELINE (PAPER'S FIG. 7)



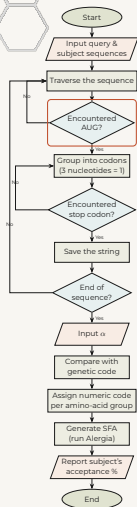
Read in the query and subject DNA/RNA sequences to be compared.

THE FULL PIPELINE (PAPER'S FIG. 7)



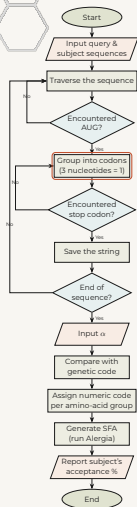
Scan through the sequence one base at a time.

THE FULL PIPELINE (PAPER'S FIG. 7)



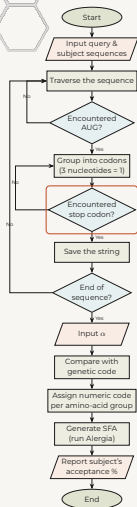
Keep scanning until the start codon AUG is found — that's where translation begins.

THE FULL PIPELINE (PAPER'S FIG. 7)



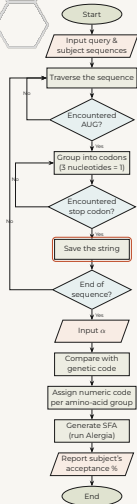
From AUG onward, group every 3 bases into one codon.

THE FULL PIPELINE (PAPER'S FIG. 7)



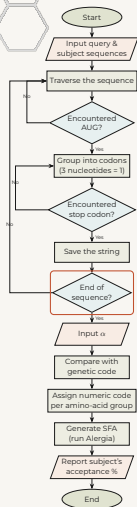
Keep grouping codons until a stop codon (UAA, UAG, or UGA) appears.

THE FULL PIPELINE (PAPER'S FIG. 7)



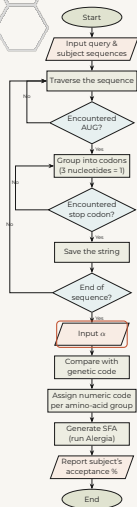
Save the completed codon string for this region.

THE FULL PIPELINE (PAPER'S FIG. 7)



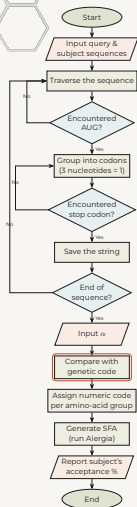
If sequence remains, go back and look for the next AUG.

THE FULL PIPELINE (PAPER'S FIG. 7)



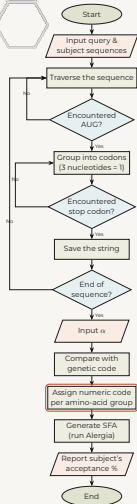
Choose the confidence parameter α , which controls how loosely states may merge.

THE FULL PIPELINE (PAPER'S FIG. 7)



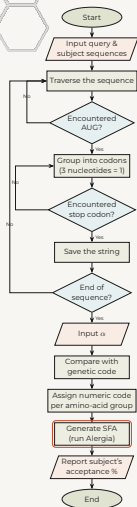
Look up each codon's amino acid using the standard genetic code table.

THE FULL PIPELINE (PAPER'S FIG. 7)



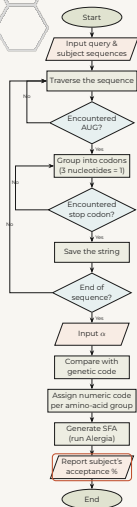
Replace each codon with its polarity-group number (0–3, or –1 for a stop codon).

THE FULL PIPELINE (PAPER'S FIG. 7)



Run the Alergia algorithm on the codon strings to build the automaton.

THE FULL PIPELINE (PAPER'S FIG. 7)

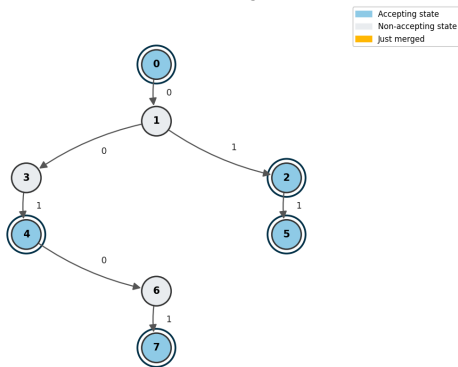


Test the subject sequence against the automaton and report its acceptance %.

- A from-scratch Python implementation of the full pipeline: PTA construction → compatibility test → recursive merge/fold
- **Verified for correctness** against the paper's own worked example (its Table 1, and its 10-string toy training set)
- Our simulation reproduces:
 - the exact same per-state frequency statistics as the paper's Table 1
 - the exact same final acceptance rate (**40%**) on the paper's own test set Q

SIMULATION: INITIAL PTA

Initial PTA (8 states) - matches Fig. 8



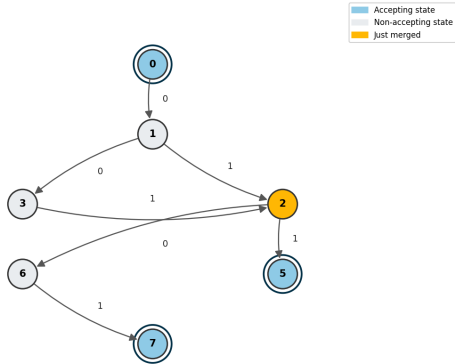
Training set:

$S = \{\lambda, \lambda, 01, 01, 001, 001, 001, 011, 00101, 00101\}$

8 states — matches the paper's Fig. 8 exactly.

SIMULATION: MERGE 1 OF 4

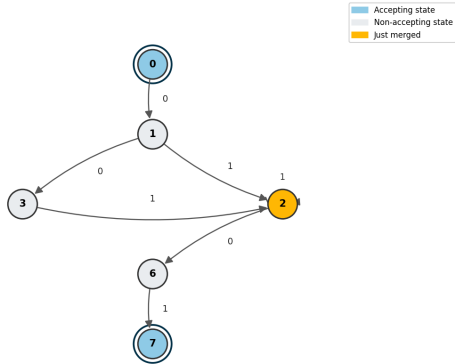
Merge 1/4: state 4 → state 2 (8 states → 7 states)



8 states → 7 states.

SIMULATION: MERGE 2 OF 4

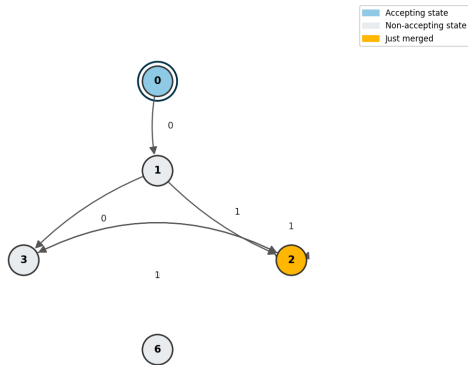
Merge 2/4: state 5 → state 2 (7 states → 6 states)



7 states → 6 states.

SIMULATION: MERGE 3 OF 4 — A CASCADE

Merge 3/4: state 7 $\rightarrow$ state 2 (6 states $\rightarrow$ 5 states)

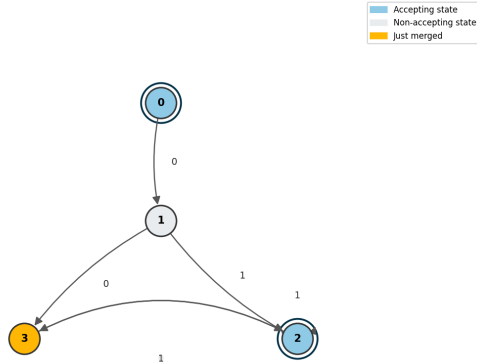


6 states $\rightarrow$ 5 states.

This merge was triggered internally while resolving Merge 4 — one compatibility decision pulled two states together at once.

SIMULATION: FINAL AUTOMATON

Merge 4/4: state 6 $\rightarrow$ state 3 (5 states $\rightarrow$ 4 states)



8 states $\rightarrow$ 4 states, in 4 real merge steps.

Acceptance rate on test set Q : **40%**
(matches the paper exactly).

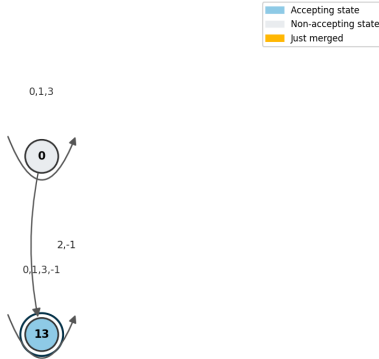
- Same pipeline, now run on codon-group strings from the paper's own DNA example (Chapter 4 training strings)
- Initial PTA: **80 states**

α	Merge events	Final states
0.7 (strict)	8	72
0.5	70	10
0.3 (loose)	78	2

Lower $\alpha \rightarrow$ dramatically more generalization.

SIMULATION: DNA AUTOMATON AT $\alpha = 0.3$

Final automaton, DNA example (alpha=0.3)



At very loose α , 80 states collapse into just 2 — almost everything becomes self-loops on one accepting state.

- These slides show discrete steps — the live demo also has two **animated** versions, played outside the slides:
- `toy_merge_animation.gif` — the 4-step example above, playing start to finish
- `dna_merge_animation.gif` — the DNA automaton actually shrinking, merge by merge, from 80 states down to 10

Built in Java (Eclipse): AminoView.java (I/O) + Train.java (algorithm logic).

Function	Role
GetCodons()	Parses DNA into codon-group symbols
CreatePTA()	Builds the initial prefix tree
Compatible() / Differ()	The statistical merge-decision test
Delta(i, t)	Transition lookup (state $\times$ symbol $\rightarrow$ state)
Combine() / MergeAll()	Performs the merges, updates statistics

- Alergia turns a rigid PTA into a compact, mutation-tolerant automaton by statistically merging equivalent states
- The confidence parameter α directly controls how much the automaton generalizes
- Our simulation reproduces the paper's own results exactly, confirming the implementation is correct
- Same pipeline applies directly to DNA codon-group sequences